

trainv6.py — task-oriented reproducible pseudocode

Reproducible algorithm edition · source SHA-256 245a44441258c6380ab5aa1d9dd36b470a3779acc8b4853e0bc296eee8c77442

This document specifies the behavior of scripts/trainv6.py at SHA-256 245a44441258c6380ab5aa1d9dd36b470a3779acc8b4853e0bc296eee8c77442 (2,149 physical source lines). It is organized by algorithms and tasks, not by the order in which incidental Python syntax appears.

The notation is Python-shaped Program Design Language: REQUIRE fails closed, WITH scopes hooks or frozen state, DETACH severs autograd, and EMIT writes an externally visible record. Each numbered step states its owning source-line range. Taken together, the detailed steps own every physical source line exactly once; comments and blank separators belong to the nearest governing step. The companion trainv6_pseudocode_map.tsv gives the exact source-line → algorithm-step and physical-pseudocode-line relation.

The RUN algorithm is a navigation overview and intentionally cross-references the detailed algorithms. The detailed A01–A17 steps are the reproducible specification and the exclusive owners used by the line map. “Reproduce” here means that an implementer can reconstruct the same observable algorithm—data identity, tensor paths, gradients, metrics, gates, files, and control flow—while ordinary Python spelling and diagnostic prose may differ.

Algorithm RUN — execute one v6 experiment

TASK Given a preset, training law, and run configuration, admit only pinned data/model topology, certify censorship and graph locality, evaluate epoch zero, train block-locally, evaluate every epoch, and save or abort with complete telemetry.

- RUN.01** Parse and validate the CLI; validate pinned artifacts; tokenize and construct the exact working item set. [calls A12, A13; source L1140–1293]
- RUN.02** Load the model, attach decoder-only LoRA, prove topology and vocabulary freezing, then create target caches, objective, taps, and optimizer. [calls A06–A09, A13; source L1295–1498]
- RUN.03** Certify the passage intervention and one-layer gradient isolation before any optimizer update. [calls A15; source L1594–1688, L1956–1957]
- RUN.04** Evaluate epoch zero through the same blocked student path and record teacher ceiling, standards, fixed-sequence distances, and generation baselines. [calls A10, A11, A16; source L1690–1954, L1959–1960]
- RUN.05** FOR each epoch: create targets, run the isolated student blocks, accumulate uniform local losses, update LoRA, emit metrics, and invoke the evaluation/stop decision. [calls A02, A14, A16, A17; source L1962–2137]
- RUN.06** Save the final adapter state and completion record, close telemetry, and report the run directory. [calls A17; source L2138–2149]

Algorithm A01 — define the experiment contract

TASK Define the two scientific laws and every immutable preset, path, schema, evaluation flag, and metric denominator needed by later algorithms.

- A01.01** [source L1–L12] DEFINE a standalone program that owns validation, intervention, training, evaluation, telemetry, and certification; each trainable block consumes detached state, learns only from teacher state, and cannot update embeddings/final norm/LM head.
- A01.02** [source L13–L18] DEFINE causal_residual: replay the same detached block input through frozen visible and blocked variants; train the adapter-on blocked variant toward the visible output.
- A01.03** [source L19–L23] DEFINE partial_teacher: form a frozen trajectory where only certified retrieval layers see passage content; train the fully blocked student toward that trajectory.
- A01.04** [source L24–L31] DEFINE full-slot censorship: keep passage slots/positions; block passage keys for later softmax queries; zero passage rows only at Qwen recurrent token mixers; use passage-removed natural positions for deployment evaluation.
- A01.05** [source L32–L49] ENABLE postponed annotations; IMPORT argparse, copy, hashlib, json, math, random, re, subprocess, time, unicodedata, contextmanager, Path; SET ROOT := resolved parent of scripts and METRIC_SCHEMA := "v6_metrics_v1".

- A01.06** [source L50–L63] PRESET gemma4_31b := {model:"google/gemma-4-31B-it", responses:"runs/vllm_h100/gemma4_31b_it/responses_bs256.jsonl", examples:"data/combined/examples_v5rs_window.jsonl", layers:60, layer_types:NONE, retrieval:{5,17,23,29,35,41,47,53}, evidence:"runs/v5_teacher_attn/layer_censor_v3.json", SHA examples:575b9dea35e0179dcdf7a513416e640db899c9bf9584236088f2921cce7a7042, responses:9449db12cbde264ebcd1bc71b169294202efcab180f2b94c2b5c0251ecceafd8, evidence:9e104757c1adc90aeb12fa71bf53141df355aa2302b7abae8b6e077637048e, summary:8e57ee12e367ad7741bc717cdb1ae8e812fdf5e378d29dcdcab099c30c1a385}.
- A01.07** [source L64–L89] PRESET qwen36_27b := {model:"Qwen/Qwen3.6-27B", responses:"runs/vllm_h100/qwen36_27b_full_exactids/responses_bs256.jsonl", examples:"data/combined/examples_v5rs_window.jsonl", layers:64, layer_types:[linear,linear,linear,full]×16, retrieval:=all indices mod4≠3 ∪ {11,15,19,23,47,51,55,59}, evidence:"runs/v5_teacher_attn/layer_censor_qwen36_v3.json", SHA examples:575b9dea35e0179dcdf7a513416e640db899c9bf9584236088f2921cce7a7042, responses:ed556e04a84cd9e07fc6411a9ae518cd8bd2d4478160d81ceb1098436609c5c8, evidence:b46b8339f08116b9290c3fc973f2072c3734f36e6b7ef9d3932cc7f5c9418fba, summary:984cc07f29136f81840eb46b485c8230b5c2234869e923975079ad8e135e74d2}.
- A01.08** [source L90–L109] SET CORPUS_PATHS:={mach:"data/poem/raw.txt", quij:"data/quijote/raw_ch4.txt"}; EVAL_ONLY:={evaluation_only:true,used_for_backward:false,optimize_weight:0}; PROFILE_SPECS: hidden_huber→(hidden_huber_profile,answer_tokens,answer_token_weighted_mean), lens_js→(lens_js_profile,answer_tokens,answer_token_weighted_mean), anchor→(anchor_profile,anchor_rows,anchor_row_weighted_mean), causal_effect_rms→(causal_effect_rms_profile,items,item_weighted_mean).

Algorithm A02 — aggregate local metrics and attribute gradients

TASK Preserve scientifically correct denominators while avoiding hot-loop synchronization, and measure each objective's gradient magnitude, share, and direction.

- A02.01** [source L110–L116] LayerMetricLedger(n): for each profile allocate n optional device-resident numerators; initialize one integer counter for each declared denominator.
- A02.02** [source L117–L121] count(tokens, anchors, items): increment answer_tokens, anchor_rows, and items by the current batch quantities.
- A02.03** [source L122–L126] add(layer, values): for each supplied metric, store value when empty else add it on device.

- A02.04** [source L127–L136] finish(): for each metric, IF values[0]=NONE return profile NONE; ELSE return [FLOAT(values[layer])/MAX(1,count[declared_denominator])] for every layer; also return each metric's aggregation label.
- A02.05** [source L137–L146] objective_gradient_attribution(objectives, parameters, device): for every objective call autograd.grad(retain_graph=true, allow_unused=true), or substitute an all-NONE tuple when objective is absent.
- A02.06** [source L147–L159] For each objective compute $\sum ||\text{gradient}||^2$ in float, norm := sqrt(sum), total_norm := MAX(sum(norms),1e-30), then DETACH norm and norm/total_norm.
- A02.07** [source L160–L170] For pairs hidden↔lens_js, hidden↔anchor, lens_js↔anchor compute dot over mutually present gradients and cosine := dot/MAX(norm_left*norm_right,1e-30); DETACH and RETURN all attribution fields.

Algorithm A03 — normalize text and derive canonical targets

TASK Reconstruct evaluation references from canonical corpora and provide deterministic word-level comparison primitives.

- A03.01** [source L171–L175] load_jsonl(path): UTF-8 parse every nonblank JSON line and RETURN the list.
- A03.02** [source L176–L186] sha256_file(path): stream binary MiB chunks into SHA-256 until EOF and RETURN hex digest.
- A03.03** [source L187–L195] corpus_lines(path): RETURN stripped nonblank lines whose left-stripped text does not begin with '#'.
- A03.04** [source L196–L209] words(text): NFC-normalize and casefold; accumulate characters until whitespace or Unicode punctuation; flush nonempty words and RETURN them.
- A03.05** [source L210–L222] words_with_blanks(text): protect every " " with a sentinel, split, restore each blank as its own token, and tokenize every other piece with words().
- A03.06** [source L223–L233] lcs_length(left,right): use a one-row dynamic program; on equality use previous_diagonal+1 else MAX(up,left); RETURN final cell.
- A03.07** [source L234–L238] lcs_score(reference,hypothesis): tokenize both and RETURN LCS/reference_length, or 0 for empty reference.
- A03.08** [source L239–L256] longest_prefix(reference,hypothesis): for each occurrence of reference[0] in hypothesis, extend contiguous equality; RETURN maximal length and its start, or (0,NONE).
- A03.09** [source L257–L267] canonical_target(example,corpora): REQUIRE known corpus and $0 \leq lo < hi \leq line_count$; SELECT canonical source lines [lo:hi].
- A03.10** [source L268–L272] IF kind∈{next,prev}, RETURN selected lines joined by newline; ELSE REQUIRE kind=cloze.
- A03.11** [source L273–L287] For cloze, split canonical span and question on whitespace; slide a canonical-length question window; accept it only if it contains " " and every pair satisfies observed=" " OR words(observed)=words(expected); REQUIRE exactly one window.
- A03.12** [source L288–L302] For each chosen pair append expected when observed=" ", otherwise REQUIRE words(observed)=words(expected); REQUIRE at least one appended token; RETURN appended canonical tokens joined by spaces.

Algorithm A04 — score generations and quantify paired change

TASK Turn item-level text into content/recitation evidence and compare checkpoints against their paired epoch-zero outputs.

- A04.01** [source L303–L312] paired_bootstrap(current,baseline,seed,draws=4000): align sorted shared IDs; if empty RETURN n=0, delta=NONE, ci95=[NONE,NONE].
- A04.02** [source L313–L323] Compute paired differences and mean; with Random(seed), repeat draws times: sample len(differences) indices with replacement and average; sort samples; lo:=samples[int(.025*(draws-1))], hi:=samples[int(.975*(draws-1))].
- A04.03** [source L324–L332] RETURN n, rounded mean delta, rounded interval, draws, and seed.

- A04.04** [source L333–L347] score_generation(item,text,count,finish): prefix,start:=longest_prefix(canonical,text); canonical_words:=words(canonical); output_words:=words(text); divergence_start:=start OR 0; derive first divergent reference word at prefix and output word at divergence_start+prefix; compute canonical and teacher-answer LCS.
- A04.05** [source L348–L380] RETURN identity/references/output, finish and lengths, six-decimal LCS values, content_exact:=(output_words=canonical_words), recitation:=(content_lcs≥.9), first_content_token_correct:=BOOL(prefix), prefix_length/fraction, reference divergence index:=prefix if in range else NONE, and output divergence index:=divergence_start+prefix if in range else NONE, with their words.
- A04.06** [source L381–L414] summarize_generation(rows).one(group): RETURN item count and six-decimal means (each divided by MAX(1,item_count)) of content LCS, teacher LCS, recitation, exactness, BOOL(prefix), and prefix fraction.
- A04.07** [source L415–L425] Group rows by corpus and kind; RETURN one(all), sorted per-corpus one(group), and sorted per-kind one(group).
- A04.08** [source L426–L446] grouped_bootstrap(rows,baseline,seed): for field in {corpus,kind}, enumerate sorted groups; align content_lcs with baseline and call paired_bootstrap(seed+group_offset+(100 if field=kind else 0)).
- A04.09** [source L447–L479] self_test_metrics(): assert Unicode tokenization, LCS/prefix, next/cloze targets, deterministic +0.1 bootstrap, and perfect generated-content scoring; PRINT PASS.

Algorithm A05 — assemble and certify training items

TASK Join generated teacher answers to canonical examples, derive the censored prompt and passage span, and reject any identity or coverage ambiguity.

- A05.01** [source L480–L491] build_items(examples,responses,tokenizer,limit=0): index examples by ID; load responses; REQUIRE exactly one non-NONE stop_token_id; load each configured canonical corpus.
- A05.02** [source L492–L496] selected:=responses; IF limit>0, selected:=responses[:MAX(1,len(responses)//limit)][:limit]; initialize items and seen IDs.
- A05.03** [source L497–L508] FOR response: REQUIRE unseen ID and matching example; decode stored prompt IDs and REQUIRE encode(decoded, no-special-tokens)=original IDs.
- A05.04** [source L509–L524] Remove the first exact privileged passage, or its stripped form; encode censored text; compute common token prefix cut and gap:=len(full)–len(censored); REQUIRE gap>0.
- A05.05** [source L525–L528] Compute canonical target and token IDs; REQUIRE nonempty canonical IDs.
- A05.06** [source L529–L543] APPEND item with IDs/corpus/kind, full/censored prompts, answer IDs/text, canonical text/IDs, passage [cut,cut+gap], position_gap, and optional teacher_word_acc.
- A05.07** [source L544–L548] IF data/combined/quij_chapter_map.json exists, load it and replace item corpus with mapping.get(example_id,current_corpus).
- A05.08** [source L549–L551] Without limit REQUIRE item count=response count; RETURN items, stop_id.

Algorithm A06 — resolve model endpoints and build tensors

TASK Abstract supported model nesting and construct the exact fixed-sequence tensors/spans consumed by training and evaluation.

- A06.01** [source L552–L575] resolve_stack(causal_lm): try model, language_model, model.language_model, language_model.model, model.model; choose object having layers and embed_tokens; resolve lm_head from outer model or language_model; REQUIRE a head; RETURN stack,head.
- A06.02** [source L576–L578] FAIL if no supported text decoder is found.

- A06.03** [source L579–L600] `collate(items, key, pad, device):` sequence:=item[key]+answer_ids; right-pad IDs/mask; answer span:=(prompt_length-1, answer_length); passage span:=stored span only for full prompt else (0,0); RETURN device tensors, spans, and lengths.
- A06.04** [source L601–L614] `slice_rows(hidden, spans)` selects row[start:stop+length]; head_logits(head, hidden, softcap) computes float logits and, if configured, softcap=tanh(logits/softcap).

Algorithm A07 — LayerwiseTaps: isolate each block's graph

TASK Intercept every decoder block so its live input is detached, while preserving enough arguments to replay that block independently and retaining its local output graph.

- A07.01** [source L615–L624] INIT with ordered layers; allocate per-layer input/output/call slots and hook handles.
- A07.02** [source L625–L634] PRE-HOOK positional case: IF args[0] is tensor, detached:=DETACH(args[0]); store detached and (args[1:], copy(kwargs)); replace args[0] with detached.
- A07.03** [source L635–L643] PRE-HOOK keyword case: IF kwargs.hidden_states is tensor, DETACH/store/replace it; save replay kwargs after removing hidden_states.
- A07.04** [source L644–L646] Otherwise FAIL because a block was called without hidden state.
- A07.05** [source L647–L651] POST-HOOK: store output[0] for tuple output, else output, without detaching the local graph.
- A07.06** [source L652–L665] WITH active(): register keyword-aware pre-hook and post-hook for every layer; YIELD; FINALLY remove every handle and clear handles.
- A07.07** [source L666–L671] reset(): replace all input/output/call slots with fresh NONE arrays.

Algorithm A08 — PassageBlocker: enforce architecture-aware censorship

TASK Prevent passage content from reaching post-passage computation in both full-attention and Qwen recurrent layers while proving every requested hook ran.

- A08.01** [source L672–L687] INIT with layers, sorted unique blocked indices, per-row passage spans, handles, and fired[index]:=0.
- A08.02** [source L688–L705] FULL-ATTENTION PRE-HOOK: locate hidden state; REQUIRE attention_mask exists and has rank 4.
- A08.03** [source L706–L714] Clone mask; FOR each nonempty [start, stop), set mask[row, ..., queries stop:, keys start: stop] to false for bool masks else dtype minimum.
- A08.04** [source L715–L718] Substitute cloned mask, increment fired[index], and RETURN modified call.
- A08.05** [source L719–L729] RECURRENT positional case: clone args[0], zero hidden[row, start: stop] for every span, increment firing, and substitute clone.
- A08.06** [source L730–L737] RECURRENT keyword case: clone kwargs.hidden_states, zero every passage slice, increment firing, and substitute clone.
- A08.07** [source L738–L742] Otherwise FAIL because recurrent layer has no tensor hidden state.
- A08.08** [source L743–L753] ENTER: for each blocked layer choose layer.self_attn/full-mask hook or layer.linear_attn/zero-content hook; FAIL on unsupported mixer.
- A08.09** [source L754–L758] Register each chosen keyword-aware pre-hook and RETURN the blocker context.
- A08.10** [source L759–L763] assert_fired(): REQUIRE every blocked index has count>0; report missed indices.
- A08.11** [source L764–L769] EXIT: remove every registered handle and clear handles.

Algorithm A09 — compute the block-local objective

TASK Measure answer-state imitation with normalized hidden Huber plus bounded Jensen-Shannon divergence through a frozen vocabulary lens, with an optional prompt anchor.

- A09.01** [source L770–L781] INIT with final norm, LM head, optional softcap, hidden/JS weights, and empty device→replica cache.
- A09.02** [source L782–L799] _replica(device): on cache miss deep-copy norm/head, remove placement hooks recursively, move to device/eval, freeze parameters, cache pair; RETURN pair.

- A09.03** [source L800–L808] components(student, teacher): scale:=MAX(RMS(teacher.float), 1e-8); hidden:=smooth_l1(student/scale, teacher/scale, beta=1).
- A09.04** [source L809–L820] Compute student logits through replica norm/head with gradient and teacher logits WITH no_grad; cast to head dtype then float; apply optional softcap to both.
- A09.05** [source L821–L833] Compute log-softmaxes/probabilities; log_mixture:=LOG(MAX(.5*(student_p+teacher_p), 1e-30)); teacher_mid:=batchmean KL(teacher||mixture); student_mid:=mean_rows Σ_vocab student_p*(student_logp-log_mixture); JS:=.5*(teacher_mid+student_mid); RETURN hidden, JS.
- A09.06** [source L834–L836] total(hidden, JS): RETURN hidden_weight*hidden + js_weight*JS.
- A09.07** [source L837–L846] anchor(student, teacher): scale by teacher RMS with 1e-8 floor and RETURN beta-1 smooth-L1.

Algorithm A10 — generate, truncate, score, and persist answers

TASK Use one greedy decoding engine for natural student, exact-path teacher, sufficient-budget, 96-token, and aligned diagnostic conditions.

- A10.01** [source L847–L861] choose_panel(items, n, seed): group by corpus; Random(seed).sample up to n from each sorted group; RETURN concatenated panel.
- A10.02** [source L862–L878] generate_rows(..., deployment_budget=96, aligned=false): eval mode; if aligned set pad:=pad_id OR EOS, else replace pad only when NONE; sufficient_budget:=deployment_budget if aligned else MAX(deployment_budget, max answer length+16).
- A10.03** [source L879–L883] WITH no_grad, iterate batch_size items, or exactly one item when aligned; width:=max selected prompt length.
- A10.04** [source L884–L891] Aligned case: use unpadded prompt/mask and position_ids := [0..cut-1] + [cut+gap..cut+gap+remaining-1].
- A10.05** [source L892–L898] Natural case: left-pad prompts to width and mark valid attention tokens.
- A10.06** [source L899–L915] Call model.generate greedily with cache, mask, sufficient max_new_tokens, stop/pad IDs, and aligned positions when present; slice off the prompt width.
- A10.07** [source L916–L927] For each result, cut before first stop ID and mark finish=stop else keep all and finish=length; decode/strip; score_generation.
- A10.08** [source L928–L930] Label sufficient row with condition/budget and append.
- A10.09** [source L931–L946] Unless aligned, limited:=usable[:deployment_budget]; limited_finish:="stop" only if original finish="stop" AND len(usable)≤deployment_budget, else "length"; decode/score/label the limited stream and append.
- A10.10** [source L947–L949] RETURN sufficient_rows, deployment_rows.
- A10.11** [source L950–L962] write_item_artifact(dir, epoch, label, rows): write eval_items_e{epoch}_{label}.jsonl; RETURN relative path, SHA-256, and count.

Algorithm A11 — evaluate fixed-sequence outputs and standard capability

TASK Compute publication-critical whole-set CE/KL and acceptance without gradients, plus normalized multiple-choice likelihood on all three vendored standards.

- A11.01** [source L963–L974] output_eval(...): on head device initialize float64 CE/KL=0, student/teacher token matches=0, exact answers=0, token_count=0.
- A11.02** [source L975–L983] For each batch collate censored prompt+teacher answer and compute student final hidden states WITH no_grad.
- A11.03** [source L984–L996] If any teacher answer state is uncached, collate full prompt, run stack WITH no_grad and adapters disabled, then cache each answer slice detached on CPU by example ID.
- A11.04** [source L997–L1005] For each item align student answer rows, cached teacher rows on device, and answer token IDs.
- A11.05** [source L1006–L1017] Traverse answer rows in chunks of 16; WITH no_grad compute soft-capped student/teacher logits and log-softmaxes; choose corresponding target IDs.
- A11.06** [source L1018–L1026] Accumulate summed NLL(student, target) and KL(teacher distribution || student distribution); record each model's argmax equality to target.

- A11.07** [source L1027–L1031] Add token matches, all-token exact-answer indicators, and answer length.
- A11.08** [source L1032–L1033] REQUIRE CE and KL did not acquire autograd graphs.
- A11.09** [source L1034–L1060] RETURN token-normalized CE/KL/acceptance, item-normalized exact rates, raw matches/counts, expected token count, dataset coverage, every-teacher-token/answer-only labels, EVAL_ONLY, no validation subset, token-weighted aggregation, and teacher-forced nonautoregressive semantics.
- A11.10** [source L1061–L1073] standard_eval(...): files:={arc_easy:data/eval/arc_easy_v1.json, arc_challenge:data/eval/arc_challenge_v1.json, hellaswag:data/eval/hellaswag_v1.json}; pad:=pad_id OR EOS; eval mode WITH no_grad.
- A11.11** [source L1074–L1088] For each task/item up to limit, tokenize prompt and each choice; create prompt+choice sequences and spans starting at prompt_length–1.
- A11.12** [source L1089–L1101] Right-pad choice sequences/masks and run one stack forward pass.
- A11.13** [source L1102–L1113] For each choice compute head logits over its prediction span and score := –cross_entropy_sum/choice_length.
- A11.14** [source L1114–L1121] Count correct iff argmax choice score equals target; store accuracy and n per task.
- A11.15** [source L1122–L1135] RETURN tasks, macro accuracy, limit, file revisions, and teacher-forced normalized-option-log-likelihood semantics with autoregressive=false.

Algorithm A12 — parse and validate invocation

TASK Convert command-line text into a complete, numerically valid experiment configuration, with a metric-self-test-only escape path.

- A12.01** [source L1136–L1169] DEFINE flags: preset; method∈{causal_residual,partial_teacher}; run name; seed=17; epochs=40; lr=1e-5; LoRA r=32 alpha=64; microbatch=1; grad_accum=4; clip=.01; weight_decay=.01; hidden=1; JS=.05; anchor=1 rows=64; eval_every=2; panel/corpus=24; generation_batch=8; standard_limit=100; attribution_every=2; limit=0; dry/preflight/self-test switches.
- A12.02** [source L1170–L1171] IF self_test_metrics, RETURN args without requiring run fields.
- A12.03** [source L1172–L1179] Otherwise REQUIRE preset, method, run_name and report missing flags through parser.error.
- A12.04** [source L1180–L1189] REQUIRE hidden>0, JS≥0, epochs/microbatch/accum/cadences/limits/generation batch>0, anchor weight/rows≥0.
- A12.05** [source L1190–L1192] RETURN args.

Algorithm A13 — admit the run and construct the trainable model

TASK Prove data provenance and architecture compatibility, attach only block-local LoRA, freeze the vocabulary stack, and initialize all run state.

- A13.01** [source L1193–L1202] main(): parse args; run/return self-test if selected; choose preset; REQUIRE run_name matches [A-Za-z0-9_-]+.
- A13.02** [source L1203–L1211] Resolve examples/responses/evidence/summary paths; REQUIRE all files exist.
- A13.03** [source L1212–L1217] Load response summary; REQUIRE summary.model=preset.model.
- A13.04** [source L1218–L1232] Load retrieval evidence; REQUIRE final and model-matched; derive expected evidenced softmax layers (all preset retrieval for Gemma, only indices mod4=3 for Qwen); REQUIRE exact equality.
- A13.05** [source L1233–L1244] Hash examples/responses/evidence/summary and REQUIRE each equals its preset pin.
- A13.06** [source L1245–L1252] Load tokenizer; build_items; without limit REQUIRE 2,071 items.
- A13.07** [source L1253–L1261] Without limit REQUIRE task counts {next:1490,prev:332,cloze:249}.
- A13.08** [source L1262–L1267] Preflight override: ≤2 items, epochs=1, eval_every=1, standard_limit≤2, panel_per_corpus=1.
- A13.09** [source L1268–L1278] Count tasks/corpora; per item enumerate nonpassage prompt positions 1..len(prompt)–2 and deterministically sample min(anchor_rows,available) using seed string v6-anchor-{id}-{seed}.

- A13.10** [source L1279–L1294] PRINT data gate with model/counts/hashes/canonical/tokenizer coverage; RETURN if dry_data.
- A13.11** [source L1295–L1317] Seed Torch; REQUIRE absent output directory; create it/open metrics.jsonl; DEFINE log(kind,fields) to add schema, pipeline=6, run_name,time, JSON-write, and flush.
- A13.12** [source L1318–L1353] Read Git HEAD and EMIT provenance: args/preset/model/method/hashes, detached-input and method-specific teacher target, depth-uniform weights, softmax/recurrent censorship, end_to_end=false, cross_block_gradient=false, frozen_vocabulary=true.
- A13.13** [source L1354–L1361] Load model bfloat16, device_map=auto, SDPA; disable use_cache.
- A13.14** [source L1362–L1372] Resolve raw stack; derive its module prefix; collect every decoder-layer torch.nn.Linear name as LoRA target; REQUIRE nonempty.
- A13.15** [source L1373–L1379] Search decoder parameters for experts/router names; REQUIRE none (dense-only).
- A13.16** [source L1380–L1400] Attach zero-dropout, no-bias CAUSAL_LM LoRA; resolve base stack/head/layers; REQUIRE num_kv_shared_layers=0 and exact preset layer count.
- A13.17** [source L1401–L1416] Classify layer as linear_attention/full_attention/unknown; REQUIRE pinned topology; for Qwen REQUIRE counts=(48,16).
- A13.18** [source L1417–L1422] device:=embedding parameter device; pad_id:=tokenizer.pad_token_id OR tokenizer.eos_token_id OR 0; softcap:=config.final_logit_softcapping or NONE; REQUIRE stack.norm exists.
- A13.19** [source L1423–L1435] REQUIRE every trainable name is lora_under.layers.; REQUIRE embeddings/norm/head have no trainable parameter; fingerprint their weight sums.
- A13.20** [source L1436–L1448] DEFINE frozen_base() as full.disable_adapter() context; DEFINE vocab_tripwire() to recompute fingerprint and REQUIRE equality.
- A13.21** [source L1449–L1469] Collect all trainable and named trainable parameters; per layer REQUIRE nonempty subset; create AdamW(lr, built-in weight_decay=0); snapshot every initial adapter on CPU.
- A13.22** [source L1470–L1499] Construct LocalObjective and LayerwiseTaps; validate retrieval indices; blocked_teacher:=all–retrieval; EMIT architecture; initialize three caches, baselines, item/destructive counters, certified=false.

Algorithm A14 — produce targets, blocked student states, and one-layer loss

TASK Materialize each training law without a cross-block graph, reuse frozen CPU caches, and compute the differentiable loss owned by one decoder layer.

- A14.01** [source L1500–L1505] frozen_targets(ids,mask,spans,blocked,selectors): WITH no_grad, adapters disabled, PassageBlocker(blocked), and taps.active, run stack and copy all tap outputs.
- A14.02** [source L1506–L1513] REQUIRE blocker fired; reset taps; RETURN layer-major selected clones outputs[layer][row,selector].
- A14.03** [source L1514–L1520] cached_frozen_rows(cache,batch,build): IF all IDs cached, RETURN layer-major cached rows moved to each layer's device.
- A14.04** [source L1521–L1527] ELSE build rows; store each item's per-layer row DETACHED on CPU; RETURN fresh rows.
- A14.05** [source L1528–L1537] student_forward: WITH every layer blocked and taps active run stack; copy outputs/inputs/replay calls; REQUIRE blockers; reset taps.
- A14.06** [source L1538–L1541] REQUIRE no captured block input requires_grad; RETURN outputs,inputs,calls.
- A14.07** [source L1542–L1556] causal_targets: WITH no_grad/adapters disabled, for each layer replay identical detached input once visible and once with only that layer blocked; REQUIRE hook.
- A14.08** [source L1557–L1570] Target rows := visible answer rows; effect per item := RMS(visible–blocked); RETURN layer-major targets,effects.
- A14.09** [source L1571–L1575] layer_loss(index): concatenate student output answer rows and target rows on student device; compute hidden,JS components.
- A14.10** [source L1576–L1590] anchor:=0; if enabled, select batch items with anchor rows, concatenate their student output rows and cached teacher rows, then compute anchor loss.

A14.11 [source L1591–L1593] total:=hidden_weight·hidden + js_weight·JS + anchor_weight·anchor; RETURN all components and total.

Algorithm A15 — certify intervention and gradient locality

TASK Before optimization, prove censorship fires, blocks passage content rather than doing nothing, is deterministic enough, and confines backward to one block.

- A15.01** [source L1594–L1606] certify_intervention(): choose first item; collate full prompt; clone IDs and replace passage span with EOS, or 0 when EOS absent.
- A15.02** [source L1607–L1613] WITH no_grad/adapters disabled run native prompt twice and retain answer states.
- A15.03** [source L1614–L1633] Run fully blocked original and replaced prompts, requiring every hook; for partial_teacher also run/require blocked_teacher hook set.
- A15.04** [source L1634–L1651] Compute repeat max-abs, blocked replacement max-abs, visible-vs-blocked RMS; REQUIRE repeat≤1e-5, invariance≤5e-3, intervention>1e-6.
- A15.05** [source L1652–L1662] EMIT intervention certification with example, errors, full/recurrent layer counts, hooks_fired=all_expected.
- A15.06** [source L1663–L1668] certify_middle(index,total): if already certified RETURN; zero grads; backward total with retain_graph.
- A15.07** [source L1669–L1682] Partition named trainable gradients into inside/outside index; REQUIRE inside nonempty and outside empty; zero grads; certified:=true.
- A15.08** [source L1683–L1689] EMIT layerwise certification with probed layer, inside tensor count, leaks=0.

Algorithm A16 — evaluate, compare, promote, or stop

TASK Funnel every completed epoch through whole-set output metrics, cadence-based standards/generation, paired evidence, checkpointing, promotion criteria, and the preregistered destructive-stop rule.

- A16.01** [source L1690–L1703] log_generation(epoch,selected,scope,...): generate censored-natural sufficient and 96-token rows; for each write artifact and summarize.
- A16.02** [source L1704–L1715] baseline_key:=suffix after "_natural_" ("sufficient" or "budget96", deliberately independent of scope); map current content_lcs by ID; if its epoch-zero baseline exists compute overall/grouped paired bootstrap with seed+epoch·1009, else empty deltas.
- A16.03** [source L1716–L1720] At epoch=0 store current score map and full rows per baseline key.
- A16.04** [source L1721–L1738] EMIT generation_eval with epoch/scope/condition/budget, natural/primary/canonical roles, summary/deltas/artifact/count, EVAL_ONLY, autoregressive greedy semantics.
- A16.05** [source L1739–L1762] If requested, WITH adapters disabled generate exact-path uncensored teacher sufficient/96 rows; write and EMIT teacher_ceiling_eval for each.
- A16.06** [source L1763–L1783] If requested, choose fixed panel, generate one-item aligned diagnostic with gapped positions, write artifact, and EMIT nonprimary/non-natural EVAL_ONLY record.
- A16.07** [source L1784–L1789] RETURN sufficient natural summary, paired delta against sufficient epoch-zero baseline, and rows.
- A16.08** [source L1790–L1801] evaluate(epoch,...): eval mode; call output_eval over current whole item set at epoch zero and every completed epoch; REQUIRE actual=expected answer tokens; EMIT teacher_output_eval.
- A16.09** [source L1802–L1808] IF epoch is neither 0, cadence multiple, nor final, RETURN false without standards/generation.
- A16.10** [source L1809–L1832] Run standard_eval; establish epoch-zero standard baseline; compute per-task, worst, macro deltas; EMIT EVAL_ONLY standard record.
- A16.11** [source L1833–L1847] At epoch 0 run whole-set student plus teacher/aligned conditions, save output baseline, print headline, RETURN false.
- A16.12** [source L1848–L1857] Later run fixed panel; candidate:=panel_delta≥0.03; full_gate:=final_epoch OR candidate.
- A16.13** [source L1858–L1868] IF full_gate run whole-set natural+aligned, ELSE retain panel evidence; save checkpoint_e{epoch}.

- A16.14** [source L1869–L1877] argmax_ratio:=student_acceptance/MAX(epoch0_acceptance,1e-30); damage:=ratio<0.8 OR worst_standard≤−0.05 OR macro_standard≤−0.03.
- A16.15** [source L1878–L1890] flat:=(delta≠NONE AND delta<.01 AND ci_low≤0≤ci_high); loss_improved:=mean_local≤.9·(epoch1_loss if non-NONE else mean_local); IF items_seen≥12000 AND loss_improved AND flat AND damage increment destructive_intervals ELSE reset it to 0.
- A16.16** [source L1891–L1907] On rows also present at epoch zero, detect new recitation and compute mean change in longest-prefix fraction.
- A16.17** [source L1908–L1942] Build/EMIT promotion criteria: content_delta≥.03, CI lower>0, argmax_ratio≥.8, worst>−.05, macro>−.03, new recitation OR prefix gain≥.03, replication pending; eligible_this_run additionally requires full_gate; final_promotion=false.
- A16.18** [source L1943–L1955] If destructive_intervals<2 RETURN false; else EMIT aborted_stop_rule, print preregistered post-12k abort, RETURN true.

Algorithm A17 — train epochs and terminate

TASK Execute depth-uniform block-local optimization, preserve exact metric denominators and attribution, call the evaluation funnel, and close the run safely.

- A17.01** [source L1956–L1964] REQUIRE intervention certification; evaluate epoch 0; initialize last_epoch=0, aborted=false, epoch1_loss=NONE.
- A17.02** [source L1965–L1979] For epoch 1..epochs: train mode; length-sort items; form microbatches; shuffle batches with Random(seed·1000+epoch); initialize ledger/attribution/grad counters; zero grads.
- A17.03** [source L1980–L1983] For each batch collate full prompt+answer with answer and passage spans.
- A17.04** [source L1984–L1992] If partial_teacher, get cached/fresh answer targets from frozen_targets blocked at all nonretrieval layers.
- A17.05** [source L1993–L1995] Run fully blocked student_forward and capture outputs, detached inputs, replay calls.
- A17.06** [source L1996–L2001] If causal_residual compute causal targets/effects; else effects:=NONE.
- A17.07** [source L2002–L2012] If anchor enabled get cached/fresh fully blocked frozen anchor rows; else anchors:=empty list per layer.
- A17.08** [source L2013–L2020] Before first update compute middle-layer loss and certify_middle.
- A17.09** [source L2021–L2031] attribute:=first batch AND (epoch=1 OR epoch mod attribution_every=0); count batch answer rows, anchor rows, and items in ledger.
- A17.10** [source L2032–L2049] For every layer compute loss; when attribute, call objective_gradient_attribution on weighted hidden, weighted JS, and weighted anchor only if enabled and differentiable.
- A17.11** [source L2050–L2060] Backward total/(layer_count·grad_accum); ledger.add answer-token-weighted hidden/JS, anchor-row-weighted anchor, and item-summed causal effect when present.
- A17.12** [source L2061–L2075] Increment items; at accumulation boundary or final batch clip global norm, accumulate its float/count, optimizer.step; if weight_decay apply parameter*=1−lr·weight_decay WITH no_grad; zero grads.
- A17.13** [source L2076–L2077] Delete batch outputs, inputs, calls, targets, anchors.
- A17.14** [source L2078–L2086] Finish normalized profiles; convert each attribution tensor dictionary to floats, preserving NONE rows.
- A17.15** [source L2087–L2097] For each named adapter parse layer index, accumulate squared CPU difference from initial snapshot, then sqrt per layer.
- A17.16** [source L2098–L2104] mean_local:=mean across layers of weighted hidden+JS+anchor; save epoch1_loss at epoch 1.
- A17.17** [source L2105–L2129] EMIT epoch timing/items/loss/profiles/attribution/deltas/mean preclip norm/weights/aggregation/depth_uniform; run vocab_tripwire; print progress.
- A17.18** [source L2130–L2137] Call evaluate; if true set aborted/last_epoch and break, else update last_epoch.
- A17.19** [source L2138–L2149] Save final checkpoint; EMIT done_aborted or done with last_epoch/items; close metrics; print status; module entry calls main().